

Consignes générales

- Lisez bien chaque consigne avant de commencer.
- Consultez le cours si nécessaire, mais écrivez vous-même chaque commande et fichier.
- Ne copiez-collez pas — tapez les commandes pour les mémoriser.
- **Chaque exercice s'appuie sur le précédent.** Ne passez pas au suivant sans avoir terminé.
- En cas de blocage, relisez le message d'erreur dans le terminal — il contient souvent la solution.

1 Partie 1 — Prise en Main de Docker

(1h30)

Objectifs — Partie 1

- Vérifier que Docker est correctement installé
- Exécuter, arrêter et supprimer des conteneurs
- Télécharger et utiliser des images depuis Docker Hub
- Écrire un **Dockerfile** pour une application Node.js
- Construire une image et la lancer en tant que conteneur

1.1 Exercice 1 — Vérification de l'installation

Exercice 1 — Docker est-il prêt ?

1. Ouvrez un terminal. Écrivez la commande permettant d'afficher la version de Docker installée, puis notez le résultat obtenu.

Commande : _____

Résultat : _____

2. Écrivez la commande permettant de vérifier que Docker Compose est disponible.

Commande : _____

3. Lancez le conteneur officiel de test de Docker. Quelle commande utilisez-vous ?

Commande : _____

Que s'est-il passé ? (Cochez ce qui est vrai)

- ☐ Docker a téléchargé une image depuis Docker Hub
- ☐ Un conteneur a été créé et exécuté

- ☐ Le conteneur tourne toujours en arrière-plan
- ☐ Le conteneur s'est arrêté après l'affichage du message

4. Écrivez la commande permettant de lister **tous** les conteneurs, y compris ceux qui sont arrêtés.

Commande : _____

1.2 Exercice 2 — Manipuler une image officielle

☰ Exercice 2 — Premier conteneur utile

Dans cet exercice, vous allez télécharger l'image officielle **Nginx** et lancer un serveur web dans un conteneur.

1. Écrivez la commande pour télécharger l'image **nginx:alpine** depuis Docker Hub.

Commande : _____

2. Écrivez la commande pour lancer un conteneur Nginx avec les contraintes suivantes :

- Mode détaché (arrière-plan)
- Nom du conteneur : **mon-nginx**
- Port 8080 de la machine redirigé vers le port 80 du conteneur

Commande : _____

3. Ouvrez **http://localhost:8080** dans un navigateur. Que voyez-vous ?

4. Écrivez la commande pour afficher les logs du conteneur **mon-nginx**.

Commande : _____

5. Écrivez les deux commandes pour d'abord **arrêter** puis **supprimer** le conteneur.

Arrêter : _____

Supprimer : _____

i Rappel — Options de `docker run`

Option	Signification
<code>-d</code>	Mode détaché (arrière-plan)
<code>-name <i>nom</i></code>	Nommer le conteneur
<code>-p <i>hôte:ctn</i></code>	Rediriger un port
<code>-e <i>VAR=val</i></code>	Définir une variable d'environnement

1.3 Exercice 3 — Créer une application Node.js à conteneuriser

☰ Exercice 3 — Préparer le projet

Créez un nouveau dossier `tp-docker-api` et placez-vous dedans.

Étape 1 : Initialiser le projet Node.js

1. Initialisez un projet Node.js avec les valeurs par défaut.
2. Installez le package `express`.

Étape 2 : Créer l'application

Créez un fichier `app.js` qui expose les routes suivantes :

Route	Méthode	Réponse attendue
/	GET	JSON avec un message de bienvenue et l'environnement (<code>NODE_ENV</code>)
/api/stagiaires	GET	JSON contenant une liste de 3 stagiaires fictifs
/health	GET	JSON { "status": "OK" }

Le port d'écoute doit être lu depuis la variable d'environnement `PORT` (valeur par défaut : 3000).

Étape 3 : Créer le fichier `.dockerignore`

Listez les fichiers/dossiers à exclure de l'image Docker. Que doit-il contenir ? (Notez-les ci-dessous)

1.4 Exercice 4 — Écrire le Dockerfile

☰ Exercice 4 — Ecrire le Dockerfile

Créez un fichier `Dockerfile` à la racine de `tp-docker-api`. Remplissez chaque instruction dans le tableau ci-dessous, puis recopiez-les dans le fichier.

#	Ce que cette instruction doit faire	Instruction Dockerfile à écrire
1	Utiliser l'image Node.js 20 (variante alpine)	
2	Définir /app comme répertoire de travail	
3	Copier package.json et package-lock.json	
4	Installer les dépendances (production seulement)	
5	Copier tout le code source	
6	Définir NODE_ENV=production	
7	Déclarer le port 3000	
8	Lancer l'application avec node app.js	

💡 Indice — Pourquoi copier package.json AVANT le code source ?

Docker construit l'image couche par couche. Si `package.json` n'a pas changé, Docker réutilise la couche mise en cache et ne réexécute pas `npm install`. Cela accélère considérablement les rebuilds.

1.5 Exercice 5 — Construire et tester l'image

☰ Exercice 5 — Build et run

1. Écrivez la commande pour construire une image nommée `tp-api` avec le tag `1.0` depuis le répertoire courant.

Commande : _____

2. Écrivez la commande pour vérifier que l'image a bien été créée.

Commande : _____

3. Écrivez la commande pour lancer un conteneur avec les contraintes suivantes :

- Mode détaché
- Nom : `tp-api-ctn`
- Port 3000 mappé vers le port 3000 du conteneur
- Variable d'environnement `NODE_ENV=production`

Commande : _____

4. Testez les trois routes dans un navigateur ou avec `curl`. Cochez ce qui fonctionne :

- ☐ GET `http://localhost:3000/`
☐ GET `http://localhost:3000/api/stagiaires`
☐ GET `http://localhost:3000/health`

5. Écrivez la commande pour entrer dans le conteneur en cours d'exécution via un shell.

Commande : _____

6. **Nettoyage** : arrêtez le conteneur, supprimez-le, puis supprimez l'image.

Arrêter : _____

Supprimer le conteneur : _____

Supprimer l'image : _____